

ISY5001 Intelligent Reasoning System

Remy: An Intelligent Knowledge Base for Meal Planning Application

Final report - Group 38

Members:

Le Chi Thanh - A0352811L

Vuong Quang Viet Tung - A0357416Y

Tran Dinh Gia Khanh - A0359890J

AIS NUS-ISS 2026

TABLE OF CONTENTS

I. Introduction **1**
II. Project Background and Market Context **2**
III. Literature Review and Market Research **2**
IV. Project Scope **4**
In scope 4
Out of scope 4
Member working scope 4
V. System Architecture **6**
Tracing and monitoring 6
VI. Data sources **7**
Data architecture and tooling 7
Nutrient - USDA FoodData Central 7
Price - NTUC FairPrice 8
Recipe – Ingredient 8
Nutrient – Price 8
Recipe - Ingredient 9
Fusing data 10
Data storing for retrieving 10
VII. LangGraph workflow **13**
Overview 13
Constraint Extraction 14
Retrieval 14
Filtering 15
Ranking 15
Planning 17
Generation 17
Architectural Trade-offs 17
VIII. Evaluation for the system **18**
Evaluation setup 18
Metrics and observed results 18
Interpretation and limitations 19
IX. Future Improvements **20**
X. References **21**

I. Introduction

Remy is a weekly meal-planning recommendation application that helps users choose recipes while considering nutrition, ingredient availability, approximate grocery cost, and dietary evidence. The project treats meal planning as a data-integration and reasoning problem. A recipe is represented through its ingredient requirements; each requirement is linked to a normalized food concept; candidate nutrition profiles are used to estimate nutrients and FairPrice evidence is attached to the product or offer that can satisfy the requirement. A generic food profile, a branded product, and a dated retail offer therefore remain separate records.

A typical user request may combine a carbohydrate range, an excluded allergen, a halal or vegetarian preference, and a daily food budget. Remy models these as explicit constraints. Its reasoning flow is to identify the ingredient occurrences required by a recipe, find compatible food candidates, verify nutrition and dietary evidence, calculate the portion-level contribution, and estimate cost when package and price information are sufficient.

Meal planning sits at the intersection of public health, household spending, online grocery shopping, and dietary preferences. A user needs more than a recipe title: they may need to know whether the meal fits a carbohydrate target, whether ingredients can be bought in suitable package sizes, whether the price is affordable, and whether a product carries the required dietary evidence. These requirements come from different datasets and operate at different levels of detail, creating the central market and engineering problem for Remy.

This project report is the final result, adapted from the original proposal. Some of the part is updated with the direct feedback from professors during the presentation for the proposal itself.

II. Project Background and Market Context

For the background, this project came straight from the demand of the development team: to lazily just have a grasp of what to cook with the best nutrients, while must suffice the current personal budget.

The Health Promotion Board's My Healthy Plate model recommends filling half the plate with fruit and vegetables, one quarter with whole-grain foods, and one quarter with meat or other protein sources. It is a practical guide rather than an individualized clinical prescription. Remy can support this kind of comparison by exposing recipe-level ingredient and nutrient summaries and allowing users to set their own constraints. (Health Promotion Board, 2014). Food is also a significant household expense. Singapore's Household Expenditure Survey 2023 reported that food accounted for 20.0% of average monthly household expenditure, while 82.0% of households made online purchases in 2023. This creates an opportunity for a meal planner that connects recipe decisions to grocery prices, provided it states whether a price is a product-level offer, an approximate reference, or an older observation. (Singapore Department of Statistics, 2023a, 2023b), highlighting the real needs for a price-aware food recommendation system.

Food-labelling rules make conservative dietary reasoning necessary. Singapore Food Agency guidance requires allergenic ingredients in prepacked foods to be declared, while precautionary allergen statements may be used for inadvertent presence. The absence of an allergen declaration in an incomplete crawler export does not establish that a product is allergen-free. A retailer dietary badge is likewise an observation that needs source and date; it is not automatically equivalent to a complete ingredient audit. (Singapore Food Agency, n.d.). Despite using different datasets than Singapore ones, we have to mention this as a ground based for our logic of the application to form the ontology behind the graph.

Remy's market problem is therefore constraint-aware meal planning using imperfect, heterogeneous, and time-sensitive food data. The project addresses it by separating recipe requirements, normalized food concepts, nutrition profiles, products, and offers; retaining provenance; and explaining why a recommendation is complete, partial, or unavailable.

III. Literature Review and Market Research

Food recommendation research supports representing meal planning as a structured constraint problem. FoodKG integrates recipes, ingredients, nutrients, food taxonomies, and provenance in a knowledge graph, and demonstrates queries involving allergies, vegan restrictions, low-sugar meals, and available ingredients. This supports Remy's use of explicit entities and relationships instead of a single unstructured recipe description. FoodKG also identifies incomplete quantities and nutrient information as practical problems that the current mapping work must manage. (Haussmann et al., 2019). Beside that, Chen et al. formulate personalized food recommendation as constrained question answering over a large food knowledge graph. Their framework combines dietary preferences, allergies, health guidelines, negation, and numerical comparisons. It supports treating carbohydrate limits, excluded allergens, halal or vegetarian preferences, and budget thresholds as inspectable constraints rather than hidden ranking features. Remy applies the same principle with deterministic status values so users can see which ingredient or evidence blocked a complete result. (Chen et al., 2021)

Those two literatures left a clear gap: Existing food knowledge-graph work concentrates on recipes, ingredients, nutrition, and generic product concepts, while health-aware recommendation work focuses on user constraints and recipe ranking. These systems generally do not model a dated

retailer offer with package quantity, current price, stock state, and a link to the exact product evidence used for a recipe calculation. Remy's proposed contribution is the explicit chain: Recipe requirement to normalized food concept, to candidate food/product, to nutrition profile, to dated retail offer, then constraint evaluation. Remy was born to tackle this problem, to combine both of the ideas to the same system that is being served through a knowledge graph of foods.

Data integration is the main technical risk when we are working on this project. USDA documents that Foundation Foods, FNDDS, Branded Foods, and other FoodData Central sources have different provenance and update schedules. A branded product should not be treated as interchangeable with a generic food profile without recording the mapping basis, while the pipeline should retain source IDs, source type, normalization version, mapping confidence, nutrient basis, package basis, and evidence timestamps. The research supports an explainable data-integration contribution by proposing a method of matching through an intermediary method of generating a confidence-based fuzzy search to traverse throughout the knowledge graph. Remy should claim that it makes the reasoning path visible and preserves uncertainty across nutrition, price, package, allergen, and halal evidence. It should not claim complete retailer coverage or guaranteed dietary safety until those evidence gaps are measured and addressed. (U.S. Department of Agriculture, n.d.)

IV. Project Scope

The project covers a price-aware, nutrition-aware recipe recommendation pipeline for the existing Remy application. The scope includes the data model, mapping pipeline, reasoning rules, explainable result statuses, and validation needed to show that a recipe can be evaluated against user constraints and building a recommender system for app to suffice the user query to get the suitable meals for planner.

In scope

- Parse recipes and ingredient occurrences, including quantities, units, alternatives, preparation qualifiers, and serving assumptions.
- Normalize ingredient concepts and map them to FoodData Central profiles while retaining source IDs, nutrient basis, and mapping confidence.
- Represent FairPrice products or listings separately from generic food concepts, with package size, unit basis, observed price, retailer, source snapshot, and freshness where available.
- Evaluate carbohydrates and other mapped nutrients, excluded allergens, halal or vegetarian evidence modes, package compatibility, and approximate budget limits.
- Distinguish whole-package purchase price from consumed ingredient cost. Consumed cost is calculated only when package and ingredient quantities use compatible bases.
- Return complete, partial, unknown, or insufficient-data states and explain which constraint could not be evaluated.
- Use the existing dbt/DuckDB or DuckLake preparation flow and Neo4j loader to materialize canonical marts and fulfillment candidates.
- Measure coverage separately for nutrition, price, package conversion, dietary evidence, partial-result reasons, and whole-recipe completeness.
- Building a microservice-based architecture for the web-application, while keeping the most viable use-cases being covered (cold-start, QnA with chatbot, etc)

Out of scope

- Medical diagnosis, treatment advice, or individualized clinical diet prescriptions.
- Guaranteed allergen-free or cross-contamination-free meals when product declarations are unavailable.
- Official halal certification by the project; the system can evaluate evidence but cannot issue certification.
- Live inventory reservation, checkout, delivery scheduling, or guaranteed promotion prices.
- Complete coverage of every retailer, FairPrice SKU, or recipe ingredient.
- Automatic inference that a blank dietary field means vegan, halal, vegetarian, allergen-free, or safe.
- A claim that the crawler alone proves a weekly budget. Budget evaluation requires a defined offer snapshot, compatible package basis, and a policy for unavailable items.
- Application on different platforms (tablets, mobile, etc)

Member working scope

Our team has three members, each of them tackles different problems in the system

Member	**Scope of work**

Le Chi Thanh	Data modelling, data lake for data preparation and DevOps for database + application
Tran Dinh Gia Khanh	Recommendation workflow, web app and system integration
Vuong Quang Viet Tung	Data crawling and preparation, frontend application development and data modelling

V. System Architecture

The system is organized into three main services:

- **Web Application**: Provides the user interface for submitting meal-planning requests and displaying the generated recommendations.
- **LangGraph Workflow Service**: Exposes a Server-sent Event (SSE) connection used by the web application. It manages the recommendation workflow, including retrieving relevant food data, applying rules and calculations, calling the language model, and generating the final response.
- **Database Service**: Stores the recipe, ingredient, nutrition, price, and other supporting food data required by the workflow.

The overall flow is straightforward: the user sends a request through the web application, the web application forwards it to the LangGraph workflow through the REST API, the workflow retrieves the required information from the database and processes the recommendation, and the final result is returned to the web application.

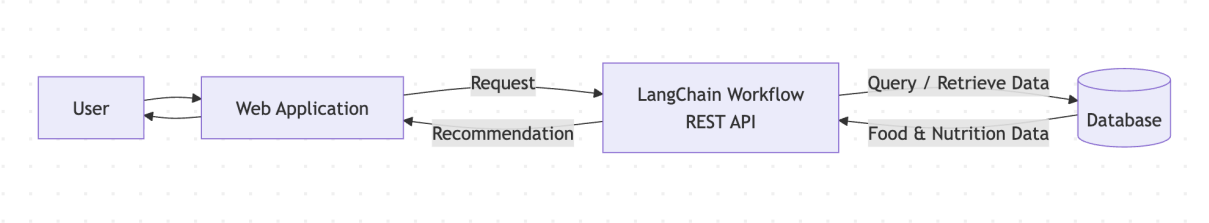


Figure 1: Workflow of the whole Remy's system

Tracing and monitoring

Langfuse provides tracing and monitoring for the recommendation service. Each meal-planning request creates a parent trace with nested spans for retrieval, filtering, ranking, solving, response generation, and judging. These spans expose stage durations, inputs and outputs, and failures, helping the team locate bottlenecks and inspect a run. The returned trace ID links a generated plan to its execution history. Ranker and data-version metadata support comparisons across changes, while nDCG@3, Precision@3, and pairwise accuracy can be attached to the same trace. Monitoring errors are logged without blocking meal-plan generation.

VI. Data sources

Data architecture and tooling

This is a chance for the team to try and test the new technologies that are being applied by the large organization into our scope of work. With the tooling for this project, team choose to deliver a defacto data warehouse using OSS or community techstack:

- Data transformation and warehousing: dbt (https://github.com/dbt-labs/dbt-core) as a build and orchestrate the data model.
- Data warehouse: duckdb as the main transformation tool (https://duckdb.org/) and ducklake (https://ducklake.select/) as the warehouse format - replacing some of the most popular techstacks recently like Spark or PostgresSQL.
- Database: Neo4j/AuraDB (https://neo4j.com/product/auradb/). We use Neo4j Community version in local and AuraDB Free Tier on the production for better collaboration with other members in the team, and the final entity graph would be served to the application through the bolt protocol of the production AuraDB instance.

The data are organized with three main layers:

- Seed: mostly the raw data files
- Staging: a one-on-one file load for data augmentation and fixing the format of some files + datatypes.
- Intermediate: some of the join, group by the edge + vertex of the final graph model

Nutrient - USDA FoodData Central

Source: https://fdc.nal.usda.gov/download-datasets - the "Full Download of All Data Types" (CSV).

Scope decision: Items labeled as branded_food type (1.89 million of the 1.92 million rows) were removed since they are brand-dependent and are not generic food components that can be used for food recipes. This leaves 24701 *laboratory-* or *survey-backed* foods (sr_legacy_food, survey_fndds_food, foundation_food, sub_sample_food and agricultural_acquisition).

Limitation: USDA is a US reference so it misses some Singaporean-specific food. However this should still have a high recall for general food components.

Columns used:

- **food.csv** - *'fdc_id'*: join key for every other file; *'data_type'*: which USDA collection the row is from, and the scope filter; *'description'*: the food name that gets normalised; *'food_category_id', 'publication_date'*: reference only
- **food_nutrient (Parquet, about 27M rows)** - *'fdc_id'* + *'nutrient_id'*: which food, which nutrient; *'amount'*: the value per 100 g; *'id'*: tie-break when a nutrient is recorded twice; *'data_points', 'derivation_id', 'median', 'footnote', 'min_year_acquired', 'percent_daily_value'*: provenance, unused
- **branded_food.csv** - *'fdc_id'*: join key; *'brand_owner', 'brand_name'*: retail identity; *'ingredients'*: the declared statement read by the allergen and vegetarian screens; *'serving_size', 'serving_size_unit', 'household_serving_fulltext', 'package_weight'*: pack context; *'modified_date'*: keeps the newest duplicate
- **food_component.csv** - *'id'*: the component; *'fdc_id'*: its parent food; *'name'*: the part (for example bone and connective tissue); *'pct_weight', 'gram_weight'*: how much of the food it is; *'is_refuse'*: inedible portion, which gives the edible-portion factor

- **nutrient.csv** - *'nutrient_id', 'name', 'unit'*: the nutrient dictionary; only the nine ids used are needed, so they are held as constants

Price - NTUC FairPrice

Source: https://www.fairprice.com.sg.

The data was crawled because there is no public API. We crawled using safe methods to avoid flooding the site. Five categories were covered, *fruits-vegetables, meat-seafood, dairy-chilled-eggs, rice-noodles-cooking-ingredients, frozen*, which gave 4683 listings, plus a targeted search crawl for items that failed to have a price after the first lookup.

Each listing carries a product id, category, name, price in SGD, selling quantity and, where applicable, a dietary attribute (Halal, Organic, etc).

FairPrice is chosen because it is Singapore's largest grocery retailer and it can represent the average price of groceries in Singapore.

Collection method - two complementary crawlers:

- A crawler crawling by category. After every scroll, only a maximum of 25 products show up more. Therefore, the scroll is being conducted until the there is nothing to scroll more (compare number of items between this scroll and the last scroll)
- A crawler crawling by searching for items on USDA that failed to match prices earlier. They are re-queried one term at a time against /search?query=<term>. Search results are server-rendered, so this runs on plain HTTP requests, and each result's own page is fetched once.

Recipe – Ingredient

Recipes: https://github.com/josephrmartinez/recipe-dataset - 13k-recipes.csv, about 13,000 recipes with title, ingredient list and instructions.

Food Component: *food_component.csv which is extracted from components of recipe.*

Columns used:

- **13k-recipes.csv** - unnamed first column: the recipe id; *'Title'*: dish name; *'Ingredients'*: the lines as published; *'Cleaned_Ingredients'*: the same lines lightly cleaned, and the column actually parsed; *'Instructions'*: method text; *'Image_Name'*: stored but unused
- **recipe_ingredient.csv (tagger output, about 148,000 rows)** - *'line'*: the original ingredient text; *'product'*: the food itself, which becomes the canonical key; *'variant', 'origin'*: qualifiers where found; *'quantity'*: the amount as written; a blank *'product'* drops the row
- **food_component.csv** - *'is_refuse', 'pct_weight', 'gram_weight'*: the edible-portion factor, shared with the nutrient side

Nutrient – Price

Objective: Give every food in the nutrition reference a Singapore retail price (from FairPrice in this case), normalised per 100 g, so nutrition and cost can be processed on the same baseline. The obstacle is that the two sides share no key: USDA writes "Pillsbury Golden Layer Buttermilk Biscuits, refrigerated dough", FairPrice writes "Malaysia Natural Fresh Japanese Cucumber". Therefore this a matching problem, which can be solved by reducing both sides to the same structure.

Preparing the nutrient side: Rule-based keyword matching (with a negation guard, so "chicken, meatless" is not counted as meat) adds pork / beef / meat flags for later dietary filtering. Finally the

free-text description is decomposed by NER into product, brand and variant spans - all spans, not just the best one, since “Bread, cheese” names two foods - while POS tagging pulls out preparation terms (frozen, toasted) in two columns: *adj* and *verb*.

Preparing the price side: The crawled listings go through the same extraction so the two sides become comparable: pack sizes normalised, country of origin resolved from a Named Entity Recognition task from a BERT model. Listings are then clustered into a product index keyed by the food itself, each entry holding its variations (origin, brand, price).

Matching: Both sides are normalised, and an inverted index is built over the nutrient product spans based on every suffix of each span. Suffix matching is crucial here: in an English noun phrase the food is the last word and the qualifiers lead, so “Pasar Japanese Cucumber” must be able to reach “Cucumber”. There are also values for columns of brand and variant as secondary matchers in case of the same key.

Pricing: Each extracted item is priced at the median of the listings sharing the product name. Pack prices are not comparable across products due to differences in measurement. This is the reason why pricing must go with quantity and unit too.

Recipe - Ingredient

Objective: Break recipes into a graph with ingredients that can be also matched with the aforementioned price - nutrient data.

Extraction: Each recipe is split into lines, with parentheticals stripped and the text lowercased. The same NER model used on the price side tags every line with product, variant, origin and quantity, one label per span. Lines whose product comes back empty fall back to POS tagging: take the noun phrase the line ends on and drop the measure words, so “16 cups watercress” gives “watercress”.

Normalisation: Numbers for quantity in various forms such as fractions or numbers are grouped into the same quantity column. Units that are used in cooking such as tsps - tsp (for teaspoon) are tracked into the column of the unit. Each occurrence keeps its characteristics such as state (raw, cooked, frozen), preparation (chopped, minced), and a choice group when the line says “or”. The following operations are applied next: strip parentheticals, drop quantities, units and filler adjectives, singularise, and apply a hash table so that scallion and spring onion both become green onion. That key is the food concept.

Mapping: The normalization processes on this and the aforementioned nutrient - gradient dataset were identical. Candidates are ranked by whether they carry a price basis and then by nutrition completeness, and are capped at five per concept so the graph stays bounded. Every concept ends with the total cost and nutritious components. Embedding search acts as the next mapper to improve the recall, strictly by the extracted entities and other characteristics.

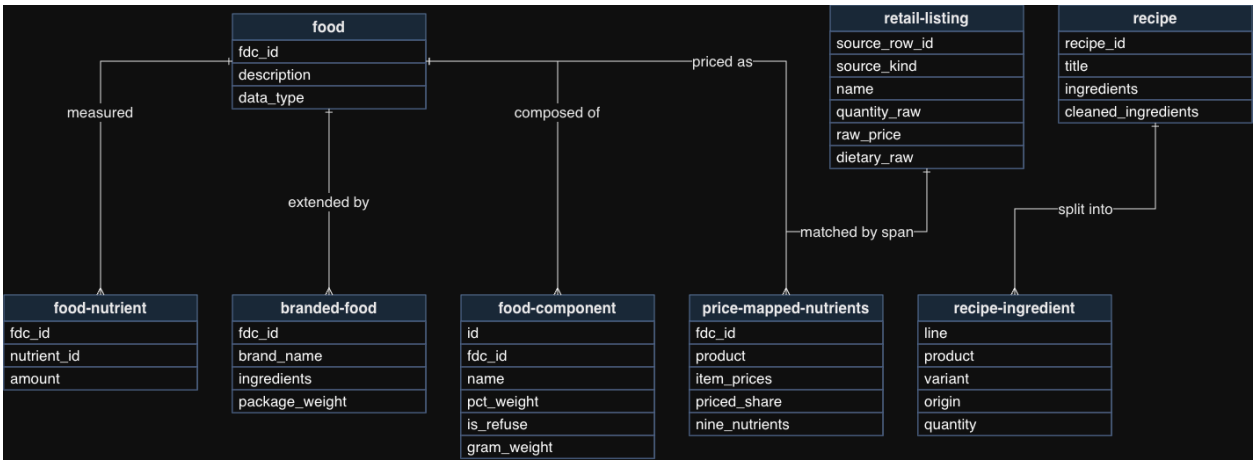


Figure 2. Mart model: a recipe requires food concepts, a food fulfills a concept with its price and screening evidence, and food components.

Fusing data

Join key: Ingredient name (through the column of 'product' across tables), materialised as the food concept. The three sources share no identifier, so each is reduced to that same normalised string and joined on it.

Source	Reduced to the value in the 'product' column	Impact on the food concept
13k-recipes	Ingredient	Quantity, unit, state, and the recipe it belongs to
USDA food	Description	Nine nutrients per 100 g, allergen and vegetarian evidence
FairPrice listing	Product	Price, pack size, halal badge

Data storing for retrieving

DT

Purpose: All prepared data lives in a DuckLake lakehouse queried by DuckDB and built by dbt. The catalogue is a SQLite file and the data itself is Parquet on disk.

There are four layers (Staging, Intermediate, Mart, Serving).

The 24 models, by layer:

Staging

Model	What it does
stg_food	USDA food rows, typed and trimmed
stg_branded_food	Manufacturer rows: brand, ingredients, pack size
stg_food_component	Food parts and edible-portion weights
stg_food_nutrient	Nutrient measurements, read straight from Parquet
stg_recipes	The 13k recipe rows
stg_price_mapped_nutrients	Crawler-priced USDA rows, with a source snapshot hash
stg_retail_listing	Aisle and search listings unioned, with lineage

Intermediate

Model	What it does
int_fdc_canonical_nutrition	Pivots the nutrient table to nine nutrients per food
int_nutrition_conflicts	Flags crawler versus USDA disagreement over one per cent
int_fdc_nutrition_profiles	Picks the winning profile and stamps its status
int_price_references	Python; parses crawler price dictionaries into span prices
int_product_identity	Parses pack size into mass, volume or count
int_price_fulfillment_candidates	Builds priced candidates from spans and listings
int_food_allergen_flags	Allergen keyword screen
int_food_vegetarian_flags	Vegetarian keyword screen

Mart

Model	What it does
node_recipe	One row per recipe: id, title, instructions
node_food_concept	Canonical ingredient names and their status
node_food	USDA foods used, with the nine nutrients
node_component	USDA food components
edge_requires	Recipe to concept, with quantity, unit and state
edge_fulfills	Food to concept, with price and screening evidence
edge_has_component	Food to component, with weights
graph_quality_manifest	Row counts against the capacity allocation

Serving

Model	What it does
recipe_graph_to_neo4j	Python; loads the seven graph relations into Neo4j

Quality control: Keys are tested for uniqueness and referential integrity and status columns for accepted values, with a set of bespoke tests guarding the contracts that matter, such as price evidence, allergen screening and the nutrition fallback. A graph quality manifest counts every node and edge table against a capacity allocation, and the Neo4j loader refuses to run if any entity is over it: the lake is the source of truth and it decides whether the graph may be refreshed.

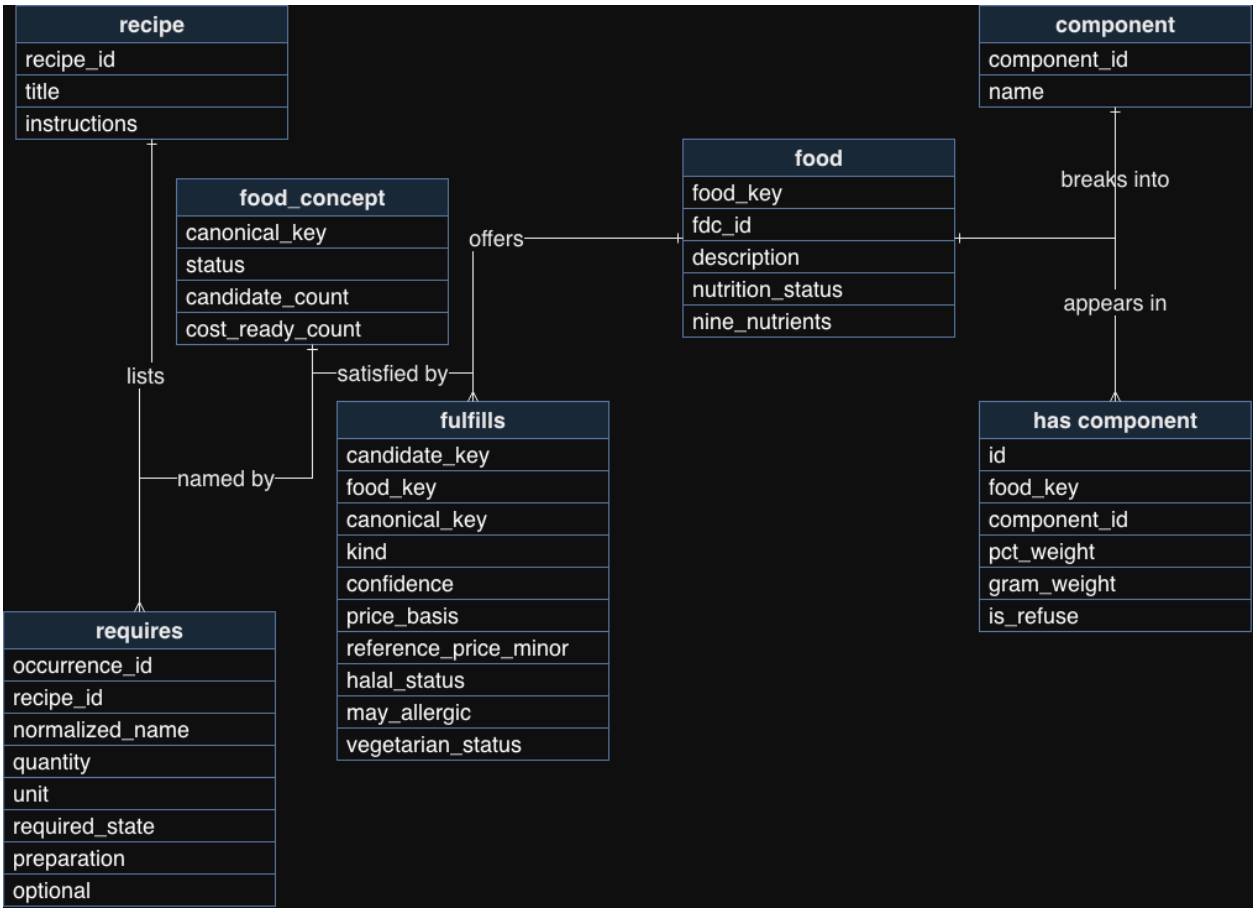


Figure 3. Entities in the lake: the USDA food tables, the FairPrice retail listings, and the recipe dataset with its tagged ingredient lines.

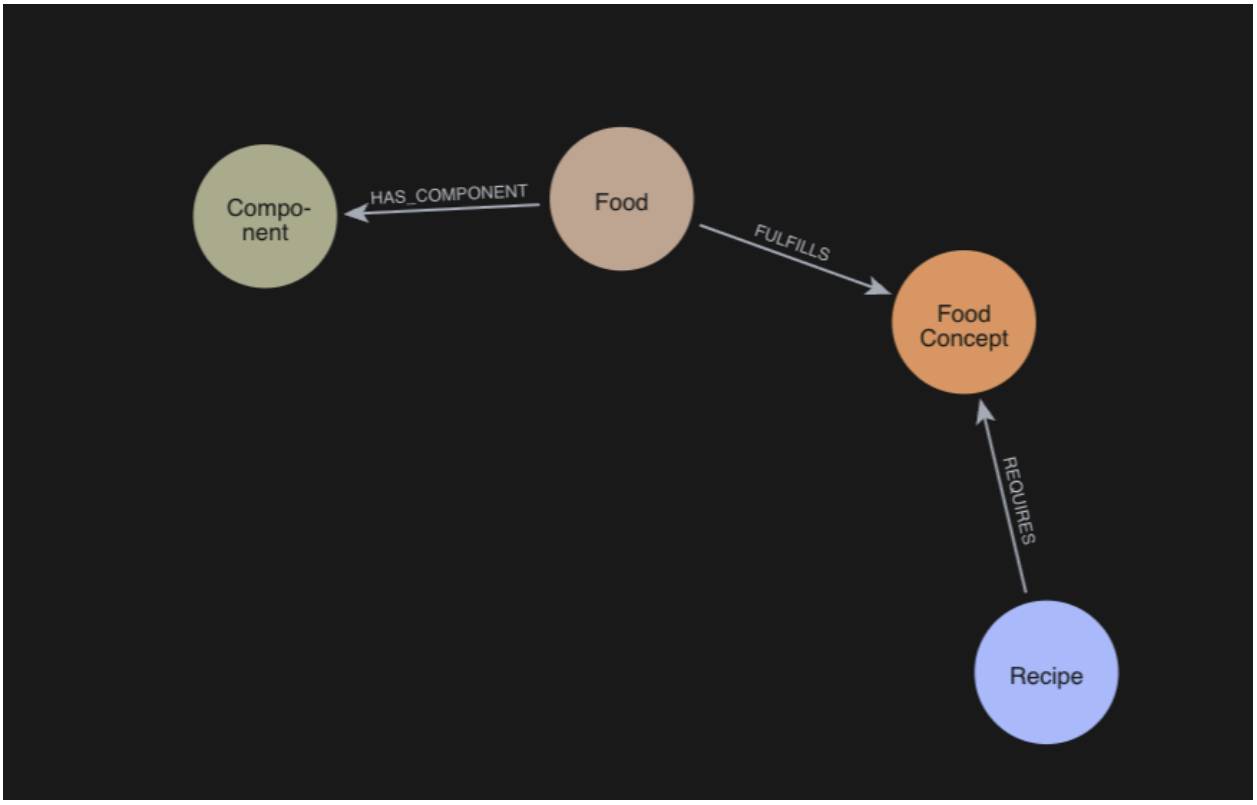


Figure 4: Schema extracted from the production database with our transformation rules

VII. LangGraph workflow

Overview

With the architecture we propose, the following LangGraph workflow would look have these steps:

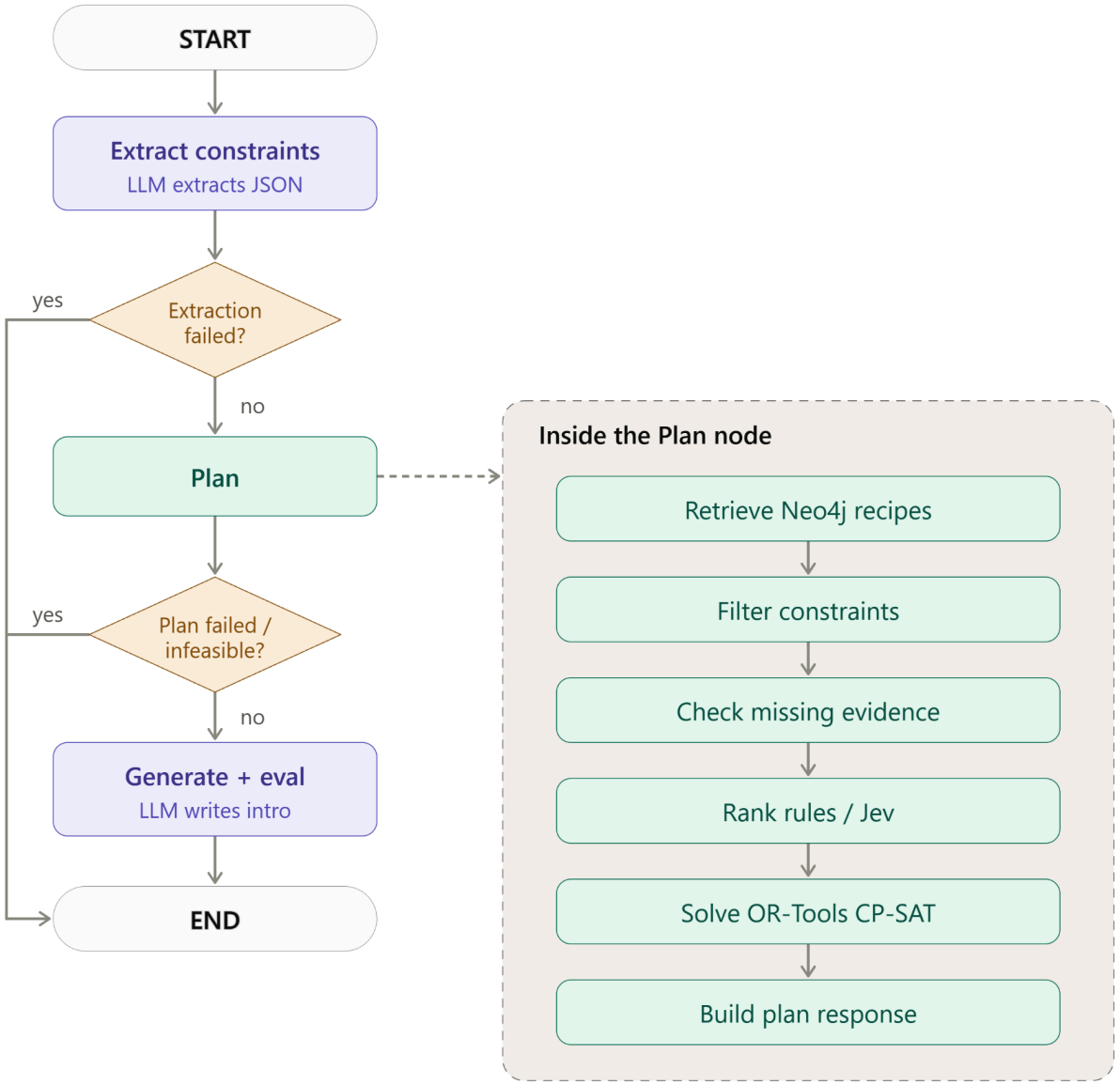


Figure 5: LangGraph workflow for Remy

The LangGraph workflow converts a user request into a set of relevant meal recommendations, wrapped inside a **three-node orchestration** (**Extract Constraints → Plan → Generate Response**). Inside the Plan node, the following deterministic pipeline executes:

- **Retrieve**: Fetches candidate recipes from the Neo4j graph using the materialized BEST_FULFILLMENT edges.
- **Filter**: Converts ingredient quantities to grams, calculates portion nutrition, and strictly drops any recipes that violate explicit allergen or dietary constraints. Absent or unknown dietary/allergen evidence must be dropped as evidence-absent (never left as "unverified" in the eligible pool).
- **Rank/rerank**: Orders filtered recipes by nutrient fit. If RANKER_MODE=jev is enabled, it reranks a shortlist by the probability that each recipe satisfies the request before selecting the final solver pool.
- **Solve**: Uses OR-Tools CP-SAT to select meals across the requested days, balancing nutrient deviations and recipe variety. If strict bounds fail, it retries with uniform nutrient-bound widening up to the user's requested tolerance. If still infeasible, it returns a hard failure naming the binding constraint.
- **Build**: Builds the structured PlanResponse: days, meals, instructions, serving-scaled ingredients/nutrients, totals, validation status, and relaxation details.
- **Judge**: This is an offline evaluation step, not part of the live request path. Scores each recipe in the ranked candidate pool from 0 to 3, then calculates nDCG@3, Precision@3, and pairwise accuracy for monitoring. This step is not shown on the UI of the application and would not affect the final state of the graph.

The core of Remy's intelligence is the plan creation pipeline. While standard AI applications often struggle with mathematical guarantees and tend to hallucinate, Remy explicitly separates deductive reasoning from probabilistic generation. This ensures that the resulting meal plan is mathematically sound, safe for the user's allergies, and fully explainable.

Constraint Extraction

The pipeline begins with Constraint Extraction, leveraging LLM Reasoning Systems. When a user submits a natural-language prompt (e.g., "Plan 3 days of meals with a severe peanut allergy, aiming for 2000 kcal"), the LangGraph orchestration workflow initiates the Extract Constraints node. We use the LLM not to generate the final answer, but as a semantic parser to convert unstructured text into a strict JSON schema. To prevent hallucination, this step is heavily constrained by deterministic regex fallbacks and strict schema validation. If a user forgets to specify a nutrient tolerance, the system conservatively defaults to 0% rather than guessing. To guarantee safety, we enforce a strict boundary between probabilistic generation and deterministic facts. The LLM is prohibited from calculating math or inventing identifiers. It acts purely as a natural language formatter for the *PlanResult* produced by the CP-SAT solver. Furthermore, a deterministic guardrail scans the generated text; if the LLM attempts to restate an *evidence-absent* recipe as selected, the system throws a validation error rather than returning hallucinated text to the user.

Retrieval

Once the constraints are extracted, the workflow moves to the Retrieval phase, which relies entirely on Knowledge Graphs. We query our Neo4j database, where recipes are not stored as flat text but are linked to FoodConcepts via REQUIRES edges, and to retail products via FULFILLS edges. By following materialized BEST_FULFILLMENT edges, the system instantly retrieves pre-calculated nutrition profiles and price evidence, avoiding the need to parse raw ingredient strings at runtime and grounding the AI in verified data.

Filtering

Before any planning happens, unsafe recipes are eliminated using Deductive Filtering. This is pure deductive logic: any recipe containing an explicit allergen, or missing evidence for a declared dietary constraint, is dropped (Rule: Absent evidence drops, never includes). We do not ask the LLM to decide if a recipe is safe; the Python code mathematically guarantees it.

Ranking

The remaining safe recipes then enter the Ranking phase. Here, the system calculates a nutrient deviation score for every recipe against the user's targets, ordering them by best fit. If Jev mode is enabled, an LLM-as-a-judge performs a reranking step to evaluate the top candidates before passing the final shortlist to the solver.

Jev-based reranking

The deterministic retrieval and filtering rules are implemented in the submitted code. Jev is used for two focused tasks: reranking candidate recipes and evaluating the resulting ranking. TypeSafe's official reranking pattern first uses a fast method to produce a shortlist, evaluates each query-candidate pair independently, and sorts the candidates by the returned probability (TypeSafe, n.d.-b).

In Remy, Jev reranking is enabled through RANKER_MODE=jev; the default mode uses nutrient rules alone. The rule ranker first orders the filtered recipes by their nutrient deviations and selects up to 2*K candidates, where K = min(50 × meals_per_day, 300). Each candidate's title, ingredients, nutrients, and instructions are sent together with the user's request to Jev. A Noul question asks whether the recipe is a good fit, with criteria requiring nutrient targets to be met within 25% and excluded ingredients to be absent. Remy makes one call per candidate, with up to eight calls running concurrently.

Jev returns a Noul value between 0 and 1: its estimated probability that the answer to the fit question is yes. This is a probability of satisfying the stated criteria, rather than a percentage measure of nutritional suitability (TypeSafe, n.d.-a). Remy sorts candidates by this value in descending order, preserves their original rule-based order for ties, and passes the top K to the meal-plan solver. The separate judge task uses a 0–3 relevance rubric to evaluate ranking quality; its grades do not determine the reranking order.

```
[{
  "recipe_id": "r_0377",
  "title": "Lentil Tomato Soup",
  "instructions": "Simmer red lentils with tomato and garlic until soft.",
  "nutrients": {
    "calories": 265.0,
    "protein_g": 14.8,
    "fat_g": 4.6,
    "saturated_fat_g": 0.6,
    "carbs_g": 41.3,
    "fiber_g": 12.1,
    "sugar_g": 8.0,
    "sodium_mg": 410.0,
    "cholesterol_mg": 0.0
  },
  "score": 0.74,
  "ingredients": [
    {
      "quantity": 150.0,
      "unit": "g",
      "text": "red lentils",
      "preparation": null,
      "optional": false,
      "position": 1,
      "occurrence_id": "r_0377-1",
      "raw_text": "150g red lentils",
      "cleaned_text": "red lentils",
      "description": null,
      "concept_name": "red lentil",
      "canonical_key": "red_lentil",
      "ingredients": null,
      "allergen_evidence_kinds": null,
      "potential_allergens": null,
      "may_allergic": false,
      "allergen_screening_status": "pass",
      "halal_status": "pass",
      "halal_evidence_kind": "ingredient_list",
      "halal_freshness_status": "fresh",
      "vegetarian_status": "pass",
      "vegetarian_evidence_kind": "ingredient_list",
      "vegetarian_concerns": null,
      "may_non_vegetarian": null,
      "nutrients": null,
      "nutrition_basis": "per_100g",
      "serving_size": 150.0,
      "serving_size_unit": "g"
    }
  ]
}]
```


Figure 6. Example of Jev reranker output schema for an items, having the probability (the score field) as the estimated distance/relevant of the input constraints/context with the meals

Planning

The ranked candidate pool is then passed to the Meal Planning step, which uses CP-SAT (Constraint Solving). We model the problem using Google OR-Tools, providing the solver with hard constraints (e.g., "must hit 3 meals a day", "avoid peanuts") and soft constraints (e.g., "try to hit 2000 kcal"). This provides a mathematical guarantee: if a valid plan exists, the solver finds it. If it is impossible, the solver throws an infeasibility error naming the exact binding constraint, rather than hallucinating a fake plan.

Generation

Finally, the solver returns the schedule, and the system enters the Structured Build & Safe Generation node. This step is a Hybrid RAG (Retrieval-Augmented Generation) approach: the LLM writes a 2-sentence friendly introduction, but the actual meal list, ingredients, and macro warnings are formatted deterministically in Python. The LLM is strictly forbidden from inventing recipe IDs or recalculating macros; it is only allowed to articulate the facts the solver produced, ensuring the final output is both human-readable and factually flawless.

Architectural Trade-offs

A key architectural trade-off was choosing a rule-based nutrient deviation ranker over a pure LLM retrieval approach. While an LLM could theoretically rank recipes by 'healthiness', it lacks the mathematical consistency and speed required for real-time planning. By using a deterministic Python ranker followed by a CP-SAT solver, we reduced planning latency to around 2 seconds, while guaranteeing that allergen constraints are never violated - something a purely LLM-driven RAG architecture cannot guarantee.

VIII. Evaluation for the system

An offline batch evaluation of 74 stored meal-planning runs produced mean nDCG@3 of 0.6928, Precision@3 of 0.6532, and pairwise accuracy of 0.6565. These results describe agreement between the recipe ranking on the recorded candidate pools. They provide an initial ranking-quality assessment within the available development sample.

Evaluation setup

The evaluation targets the ranked candidate pool produced before meal selection by the solver. Each recipe is assessed against the request's nutrient targets and ingredient restrictions using four relevance levels: 0 (unsuitable), 1 (marginal), 2 (relevant), and 3 (highly relevant). Jev's floating-point scores are rounded half up to integer grades before metric calculation. The recorded judge version was jev-judge-fit-score-v1 and the model identifier was jev-latest. These are model-generated relevance judgments; no human grades were supplied at the point. This is how the modern architecture for evaluation was built on.

Metrics and observed results

nDCG@3: 0.6928 across 74 runs. This measures the quality and ordering of the first three recommendations relative to the best possible top three from the same fully graded candidate pool.

Precision@3: 0.6532 across 74 runs. A recipe is relevant when its rounded grade is at least 2. The metric is the number of relevant recipes among the first three divided by three. The observed mean corresponds to approximately 1.96 relevant recipes per top-three list.

Pairwise accuracy: 0.6565 across 73 defined runs. This measures the fraction of recipe pairs with different grades that place the higher-graded recipe first. Tied grades are excluded. One run had no unequal-grade pairs, so its pairwise accuracy was undefined and excluded from the mean rather than counted as zero.

Figure 6 shows the per-run distributions and arithmetic means. Each run has equal weight in these means; pairwise accuracy is averaged over runs with a defined value rather than pooled across all recipe pairs.

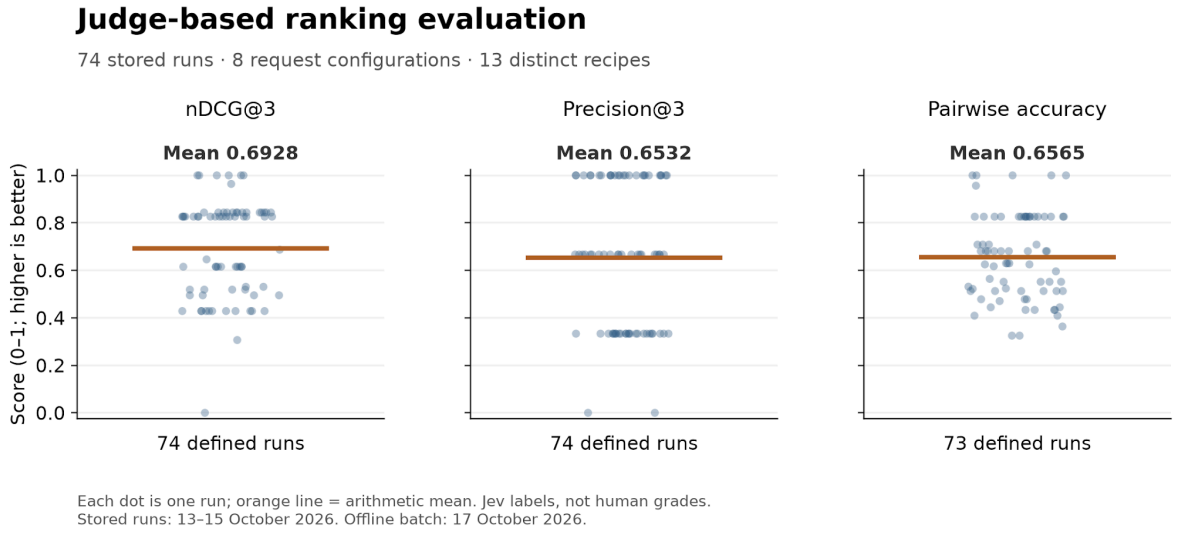


Figure 7. Jev judge metrics for the stored ranking evaluations. Dots represent individual runs and orange lines show the arithmetic means. Scores range from 0 to 1, with higher values indicating better agreement with the judge.

Interpretation and limitations

The observed top-three precision indicates that about two of the first three recipes were judged relevant on average, while nDCG@3 shows room to improve which relevant recipes receive the highest positions. Pairwise accuracy of 0.6565 describes ordering agreement across unequal-grade pairs in the evaluated pools.

For the rule-ranker subset alone, the means were 0.6891 for nDCG@3, 0.6575 for Precision@3, and 0.6523 for pairwise accuracy. A single Jev-reranked run is insufficient for a reliable comparison between ranking methods. The judge labels also require calibration against independent human review, and the jev-latest model alias does not identify an immutable model release. A stronger evaluation would use more recipes, a balanced set of held-out requests, human relevance judgments, and both ranking methods evaluated on identical candidate pools. Separate checks are needed for final-plan constraints and dietary evidence.

IX. Future Improvements

Future development should focus on making recommendations more reliable, broadening the supporting data, and evaluating the complete meal plan. The current ranking results provide a useful starting point, but improvements should be measured against a consistent baseline and supported by evidence at each stage of the workflow.

The next evaluation should use a larger and more balanced set of held-out requests covering nutrient goals, dietary preferences, allergies, and combinations of constraints. Independent reviewers should grade recipes without seeing the ranking order, allowing the Jev judge to be checked against human judgments. The rule ranker and Jev reranker should then be compared on identical candidate pools using nDCG@3, Precision@3, and pairwise accuracy. Reporting results by request type would make weaknesses easier to identify than a single overall average.

Ranking relevance should be complemented by checks on the meals selected by the solver. The implementation should distinguish non-negotiable constraints from preferences that may be relaxed. Nutrient limits currently contribute penalties to the optimization objective; limits intended to be mandatory should instead be enforced explicitly. Nutrient totals should be checked consistently against the requested servings, and any relaxation should identify the affected constraint and its effect on the returned plan. Evaluation should also report constraint violations, plan feasibility, and recipe repetition.

X. References

Chen, J., et al. (2021). Personalized food recommendation as constrained question answering over a large-scale food knowledge graph. *arXiv*.

Haussmann, S., et al. (2019). FoodKG: A semantics-driven knowledge graph for food recommendation. In *Proceedings of the International Semantic Web Conference*.

Health Promotion Board. (2014). *Health Promotion Board introduces My Healthy Plate to inculcate healthy eating habits among Singaporeans*.

Singapore Department of Statistics. (2023a). *Average monthly household expenditure among resident households*. HES 2023 data table.

Singapore Department of Statistics. (2023b). *Proportion of households with online expenditure*. HES 2023 data table.

Singapore Food Agency. (n.d.). *Food allergy and intolerance*.

Google. (n.d.-b). CP-SAT Solver. Google OR-Tools

TypeSafe. (n.d.-a). *Noul*.

TypeSafe. (n.d.-b). *Re-ranking*.

U.S. Department of Agriculture. (n.d.). *FoodData Central: Data documentation*.

Course module mapping (NUS-ISS course materials)

The AI/ML concepts used in this report map to the following course modules:

Concept in this report	**Module**
Recommender systems (recipe/meal recommendation, cold-start)	Reasoning System
Knowledge graph and graph traversal (FoodKG, constrained QA over a food KG)	Machine Reasoning
Graph database and graph-based retrieval (Neo4j)	Reasoning System
Rule-based reasoning (rule-based retrieval, keyword rules with negation guard, rule ranker)	Machine Reasoning
Ranking and re-ranking (nutrient-fit ranking, reranking of the shortlist)	Reasoning System
Semantic modelling and search improvement	Cognitive Systems
Hybrid reasoning architecture (rules + solver + LLM)	Reasoning System

Explainability (explainable result statuses, visible reasoning path)	Reasoning System; Machine Reasoning
Chatbot Q&A with MCP	Cognitive Systems; Reasoning System

AI usage

The team mainly uses a lot of AI subscriptions (Kiro, Codex, Claude) on the steps of brainstorming, data exploration and often working on the first version of our services. The first versions are often one-shot with the detail prompts written by humans, plus inject with the proposal (as a direction for the system that is being adjusted by us too). All of the prompts are the finalized result from our discussion and considering for all of the trade off by the team.

Requirements are defined in the repository prompts listed below:

Component	**Prompt path in repository**
Web frontend	Miscellaneous/component-prompts/01-web-frontend.md
Account and user-data backend	Miscellaneous/component-prompts/02-account-backend.md
Recommendation API	Miscellaneous/component-prompts/03-recommendation-api.md
Conversational application	Miscellaneous/component-prompts/04-chat-application.md
Conversational workflow	Miscellaneous/component-prompts/05-conversational-workflow.md
Shared domain	Miscellaneous/component-prompts/06-shared-domain.md
Data collection	Miscellaneous/component-prompts/07-data-collection.md
Data preparation	Miscellaneous/component-prompts/08-data-preparation.md
Neo4j graph	Miscellaneous/component-prompts/09-neo4j-graph.md
Application PostgreSQL	Miscellaneous/component-prompts/10-application-postgres.md
DuckDB and DuckLake	Miscellaneous/component-prompts/11-duckdb-ducklake.md
Observability and optional evaluation	Miscellaneous/component-prompts/12-observability-evaluation.md
System integration	Miscellaneous/component-prompts/13-system-integration.md

Resources

- Repo: https://github.com/2na3k/Workshop-Project-Submission-2026-10-10-AIS
- Video demo: https://drive.google.com/drive/folders/1fl9UMKZI6dDgRCqXvL5dvR-TUpYF5k-l?usp=sharing